Use case

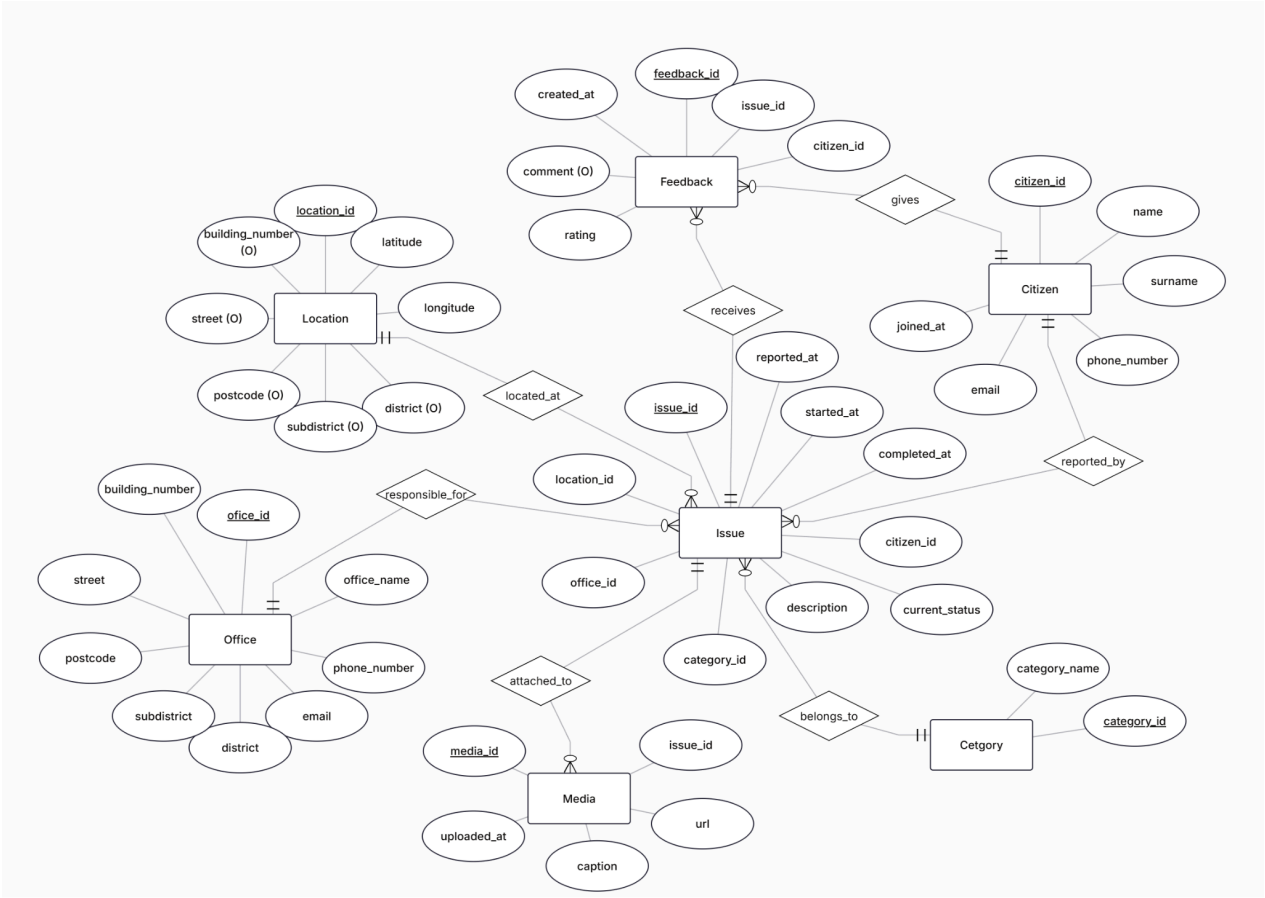
Citizen Sourcing-based System for Smart City and Smart Governance of Pathumthani (CSSS).

Online implemented Graph DB:
NEO4J_URI=neo4j+s://e10a8ae9.databases.neo4j.io
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=cyIkTvy7g9GDXrRsx94IVPXSJulFbbnjD2PsSWBscVc
NEO4J_DATABASE=neo4j
AURA_INSTANCEID=e10a8ae9
AURA_INSTANCENAME=Instance01

Recap

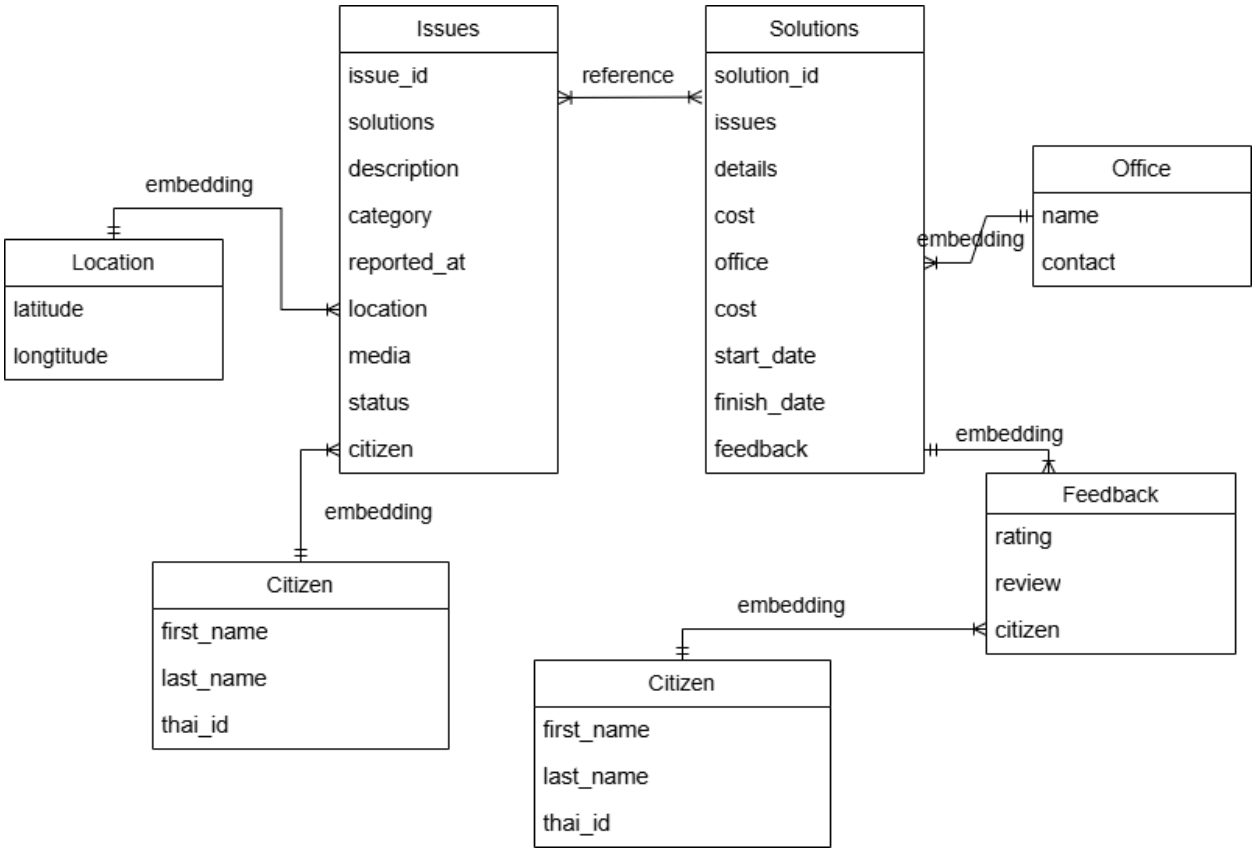
SQL

Each reported issue is considered to be a separate problem, even if many issues are related to the same real-world problems.



Document model

We added Solutions, so that we can group similar issues reported by citizens.



Graph model

Properties

Nodes

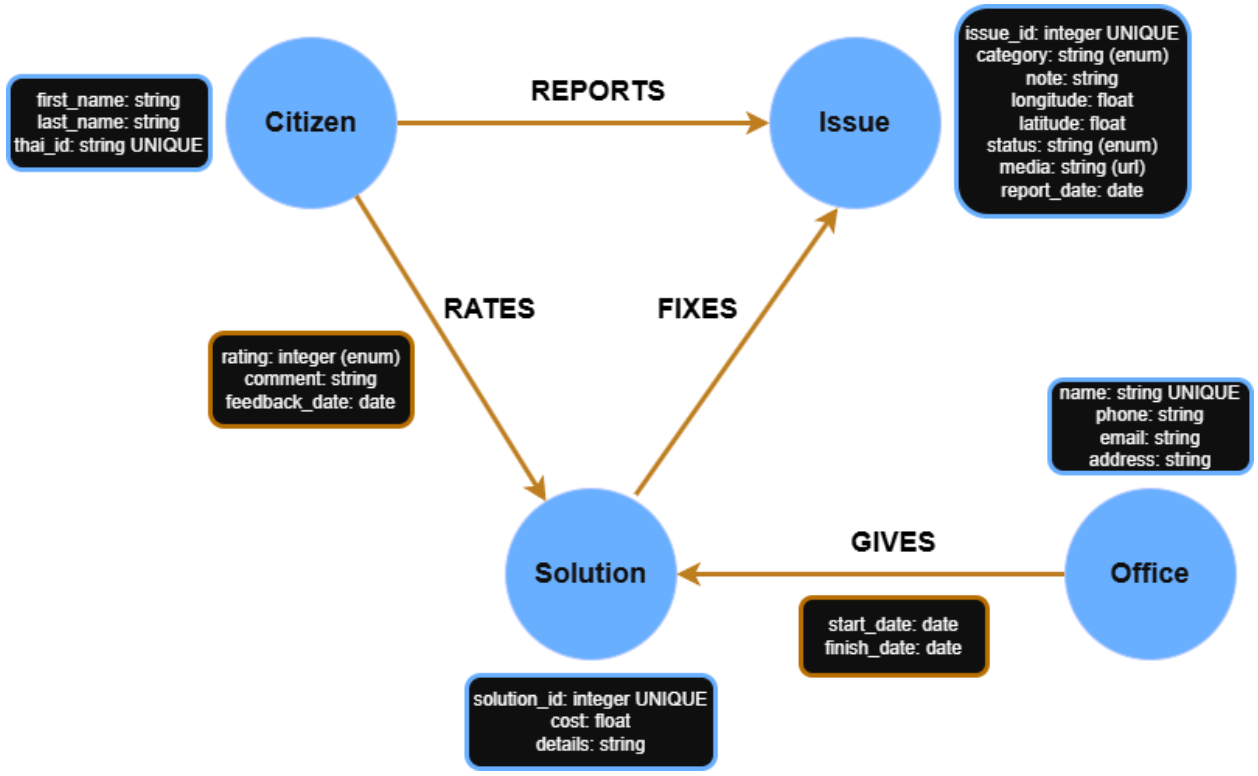
- Citizen
  - first_name: string
  - last_name: string
  - thai_id: string UNIQUE
- Issue
  - issue_id: integer
  - report_date: date
  - category: string
  - note: string
  - longitude: float
  - latitude: float
  - status: string (pending, in_progress, resolved)

    - media: string (url link)
- Solution
    - solution_id: integer
    - cost: float (will not put it until offices finish solutions)
    - details: string
- Office
    - name: string UNIQUE
    - phone: string
    - email: string
    - address: string

Relationships

- REPORTS (Citizen → Issue)
- GIVES (Office → Solution)
    - start_date: date
    - finish_date: date (no finish_date given if the solution hasn't been finished yet)
- FIXES (Solution → Issue)
- RATES (Citizen → Solution)
    - rating: integer (1-5)
    - comment: string
    - feedback_date: date

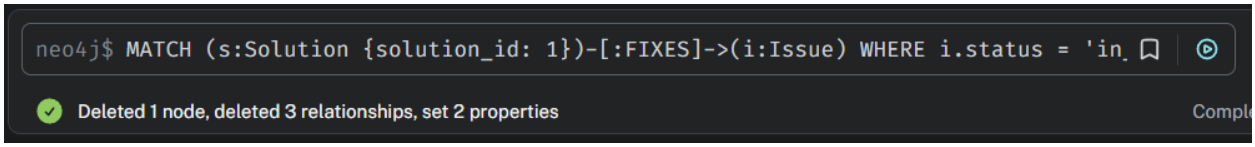
Diagram



Queries

1. **The government rejected a new solution (solution_id = 1) - delete solution and change status of all related issues from “in_progress” to “pending”.**

```
MATCH (s:Solution {solution_id: 1})-[:FIXES]->(i:Issue)
WHERE i.status = 'in_progress'
SET i.status = 'pending'
WITH s
DETACH DELETE s;
```



2. **Show yearly statistic of closed solutions per month: number of solutions, average fix time, avg solutions ratings, avg costs**

```
MATCH (s:Solution)-[g:GIVES]-(o:Office)
WHERE g.finish_date IS NOT NULL AND g.finish_date.year = 2025

OPTIONAL MATCH (s)<-[r:RATES]-(c:Citizen)
WITH s, g, avg(r.rating) AS avgRating

WITH s, g, avgRating, duration.between(g.start_date, g.finish_date).days AS
fixTimeInDays
WITH g.finish_date.month AS month,
    count(s) AS totalSolutions,
    avg(fixTimeInDays) AS avgFixTimeDays,
    avg(avgRating) AS solutionAvgRating,
    avg(s.cost) AS avgCost

RETURN month,
      totalSolutions,
      round(avgFixTimeDays * 100.0) / 100.0 AS avgFixTimeDays,
      round(solutionAvgRating * 100.0) / 100.0 AS solutionAvgRating,
      round(avgCost * 100.0) / 100.0 AS avgCost
ORDER BY month ASC;
```

	month	totalSolutions	avgFixTimeDays	solutionAvgRating	avgCost
1	8	5	0.2	4.4	8400.1
2	9	5	0.6	4.67	6050.0
3	10	5	0.8	3.33	2940.0

3. Count the number of proposed solutions that are still unfinished per office

```
MATCH (o:Office)-[g:GIVES]->(s:Solution)
WHERE g.finish_date IS NULL
RETURN o.name AS office, count(s) AS totalUnfinishedSolutions
ORDER BY totalUnfinishedSolutions DESC;
```

	office	totalUnfinishedSo
1	"Pathum Thani City Hall"	2
2	"Khlong Luang District Office"	2
3	"Lam Luk Ka Road Hazard Center"	2
4	"Pathum Thani Provincial Electricity Authority"	2
5	"Thanyaburi Police Station"	1
6	"Pathum Thani Provincial Transport Office"	1
7	"Mueang Pathum Thani Public Health Office"	1

5. Find 5 'best value' solutions (costs below average and ratings above 3)

```
MATCH (s:Solution)
WHERE s.cost IS NOT NULL
WITH avg(s.cost) AS globalAvgCost

MATCH (sol:Solution)<-[r:RATES]-(c:Citizen)
WITH globalAvgCost, sol, avg(r.rating) AS avgRating, sol.cost AS cost
WHERE avgRating > 3 AND cost < globalAvgCost

RETURN sol.solution_id,
    sol.description,
    cost,
    round(avgRating * 100.0) / 100.0 AS avgRating,
    round(globalAvgCost * 100.0) / 100.0 AS globalAverageCost
```

ORDER BY cost ASC, avgRating DESC;

	sol.solution_id	sol.description	cost	avgRating	globalAverageCos
1	22	"Inspector visited store, confirmed removal of expired goods, issued warning"	750.0	5.0	5796.7
2	20	"Investigated source, issued fine for illegal burning"	1000.0	4.0	5796.7
3	17	"Special pickup team dispatched to remove bulky item"	1500.0	5.0	5796.7
4	27	"Plainclothes officers assigned to monitor market"	3000.0	5.0	5796.7
5	14	"Recalibrated traffic signal timing during peak hours"	5000.0	4.0	5796.7

5. Find all pending issues within a specific geographic area: latitude between 13.98 and 14.00, longitude between 100.61 and 100.63 and count them.

```
MATCH (i:Issue)
WHERE i.status = 'pending'
  AND i.latitude > 13.98 AND i.latitude < 14.00
  AND i.longitude > 100.61 AND i.longitude < 100.63
RETURN count(i) AS issuesInArea;
```

	issuesInArea
1	2

6. Calculate average time to open an issue after being reported per office (group issues for each solution, count time between the oldest reported issue and the start_date of proposed solution)

```
MATCH (o:Office)-[g:GIVES]->(s:Solution)
WHERE g.start_date IS NOT NULL
MATCH (s)-[:FIXES]->(i:Issue)
WHERE i.report_date IS NOT NULL
WITH o, s, g, min(i.report_date) AS oldest_report_date
WITH o, duration.inDays(date(oldest_report_date), date(g.start_date)).days AS
open_days
RETURN
   o.name AS office_name,
   round(avg(open_days), 2) AS average_reaction_time_days,
   count(o) AS total_solutions_analyzed
```

ORDER BY average_reaction_time_days ASC

office_name	average_reaction_	total_solutions_ar
"Pathum Thani Provincial Transport Office"	1.0	5
"Pathum Thani Provincial Waterworks Authority"	1.0	2
"Khlong Luang District Office"	1.4	5
"Pathum Thani City Hall"	18.25	4
"Lam Luk Ka Road Hazard Center"	23.0	3

7. Get the average solutions ratings per office for the last 3 months

```
MATCH (o:Office)-[:GIVES]->(s:Solution)<-[r:RATES]-(c:Citizen)
WHERE r.feedback_date IS NOT NULL
  AND date(r.feedback_date) >= date() - duration('P3M')
WITH o, avg(r.rating) AS avg_rating, count(r) AS total_ratings
RETURN
  o.name AS office_name,
  round(avg_rating, 2) AS average_rating_last_3_months,
  total_ratings
ORDER BY average_rating_last_3_months DESC
```

	office_name	average_rating_la	total_ratings
1	"Lam Luk Ka Road Hazard Center"	5.0	1
2	"Khlong Luang District Office"	5.0	1
3	"Mueang Pathum Thani Public Health Office"	5.0	1
4	"Thanyaburi Police Station"	5.0	1
5	"Pathum Thani Provincial Waterworks Authority"	4.0	2

8. Find the office whose solutions have the lowest ratings

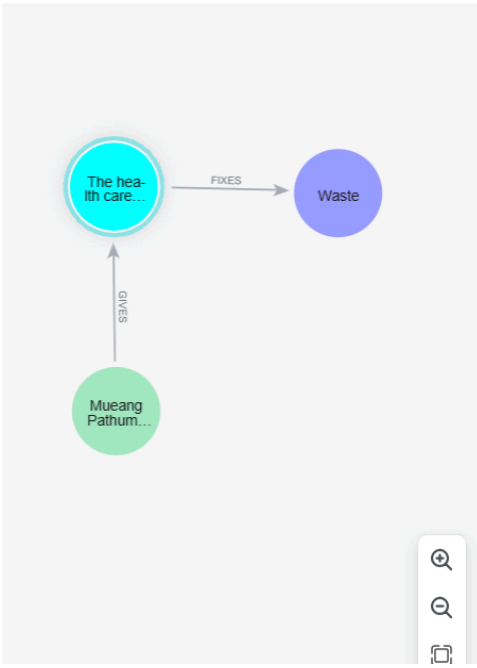
```
MATCH (o:Office)-[:GIVES]->(s:Solution)<-[r:RATES]-(:Citizen)
WHERE r.rating IS NOT NULL
WITH o, avg(r.rating) AS avg_rating, count(r) AS total_ratings
RETURN
  o.name AS office_name,
  round(avg_rating, 2) AS average_rating,
  total_ratings
ORDER BY average_rating ASC
LIMIT 1
```

	office_name	average_rating	total_ratings
1	"Pathum Thani Provincial Electricity Authority"	3.0	1

9. Office opens issue - creates solutions for related issues (change issues' status from pending to in_progress)

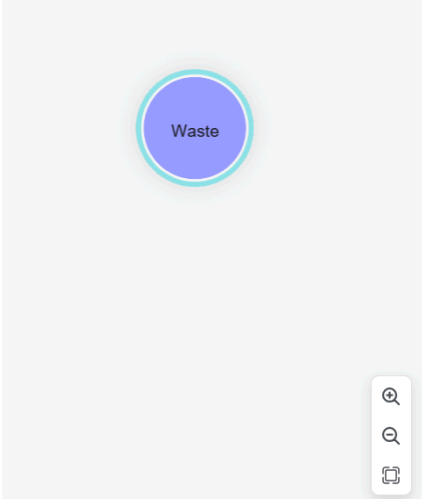
```
MATCH (o:Office), (i:Issue)
WHERE i.issue_id = 1
  AND o.name = "Mueang Pathum Thani Public Health Office"
MERGE (s:Solution {solution_id: 26})
SET
  s.description = "The health care team will go to that area to check and manage to solve the problem"
MERGE (o)-[g:GIVES {start_date: date()}]->(s)
```

MERGE (s)-[f:FIXES]->(i)
RETURN o, g, f, i, s



Key	Value
<id>	4:cb879d2a-dfc1-4d3e-b77f-39b220906a3f:88
cost	0.0
description	"The health care team will go to that area to check and manage to solve the problem"
solution_id	26

Update status from pending to in_progress:
MATCH (i:Issue)
WHERE i.issue_id = 1
SET i.status = 'pending', i.status = 'in_progress'
RETURN i



issue_id	1
latitude	13.9904
longitude	100.6119
media	"https://example.com/images/bin1.jpg"
note	"Overflowing bins near Future Park Rangsit"
report_date	2025-01-01
status	"in_progress"

10. A citizen name “Nipa”wants to remove feedback for Solution_id
MATCH (c:Citizen)-[r:RATES]->(s:Solution)
Where c.first_name = 'Nipa'AND c.thai_id = "1100701234600"
DELETE r

Deleted 1 relationship

```
MATCH (c:Citizen)-[r:RATES]->(s:Solution)
Where c.first_name = 'Nipa'
RETURN c,r,s
```

No changes, no records

Comp

Meta

11. Office finishes solution (UPDATE) - change issues' status from in_progress to resolved

First Run: :param {solution_id: 1}

And then run:

```
MATCH (o:Office)-[g:GIVES]->(s:Solution)-[:FIXES]->(i:Issue)
WHERE s.solution_id = $solution_id AND i.status = 'in_progress'
SET g.finish_date = date()
SET i.status = 'resolved'
RETURN s.solution_id AS solutionId,
    s.description AS solutionDescription,
    i.issue_id AS issueId,
    i.note AS issueNote,
    i.status AS newStatus,
    g.finish_date AS finishDate
```

Table RAW

	solutionId	solutionDescriptio	issueId	issueNote	newStatus	finishDate
1	1	"Dispatched road crew for tempora ry asphalt patc h"	16	"Deep pothole on Phahon Yothin Rd outbound near Kh long Nueng"	"resolved"	2025-11-11
2	1	"Dispatched road crew for tempora ry asphalt patc h"	17	"Another report for the same pot hole on Phahon Y othin Rd"	"resolved"	2025-11-11

12. Identify 5 solutions that haven't been finished for the longest time

```
MATCH (o:Office)-[g:GIVES]->(s:Solution)
WHERE g.finish_date IS NULL
RETURN s.solution_id AS SolutionID,
    s.description AS Description,
    g.start_date AS StartDate
ORDER BY g.start_date ASC
LIMIT 5
```

Table RAW

	SolutionID	Description	StartDate
1	3	"Contacted waste management contractor for immediate pickup"	2025-04-18
2	8	"Forwarded report to responsible construction site manager"	2025-05-21
3	9	"Contacted local internet service provider to fix cable"	2025-05-22
4	10	"Engineering team scheduled for signal diagnosti	2025-06-23

13. Find the office that proposed and finished the highest number of solutions this year

```
MATCH (o:Office)-[g:GIVES]->(s:Solution)
WHERE g.finish_date IS NOT NULL
    AND g.finish_date.year = date().year
WITH o.name AS OfficeName, count(s) AS FinishedSolutions
WITH max(FinishedSolutions) AS maxCount
MATCH (o:Office)-[g:GIVES]->(s:Solution)
WHERE g.finish_date IS NOT NULL
    AND g.finish_date.year = date().year
WITH o.name AS OfficeName, count(s) AS FinishedSolutions, maxCount
WHERE FinishedSolutions = maxCount
RETURN OfficeName, FinishedSolutions
```

Table RAW

	OfficeName	FinishedSolutions
1	"Pathum Thani Provincial Transport Office"	4

14. Find a solution that has a rating average less than 2.5

```
MATCH (c:Citizen)-[r:RATES]->(s:Solution)
WITH s, avg(r.rating) AS AverageRating
WHERE AverageRating < 2.5
RETURN s.solution_id AS SolutionID,
```

```
    s.description AS Description,
    AverageRating
```

Table RAW

SolutionID	Description	AverageRating
26	"Met with bus operator to review schedule adherence"	2.0

15. The government allocates a budget of cost per solution is 20,000 baths - find solutions that cost more than 20,000 baths

```
MATCH (s:Solution)
WHERE s.cost > 20000
RETURN s.solution_id AS SolutionID,
    s.description AS Description,
    s.cost AS Cost
```

Table RAW

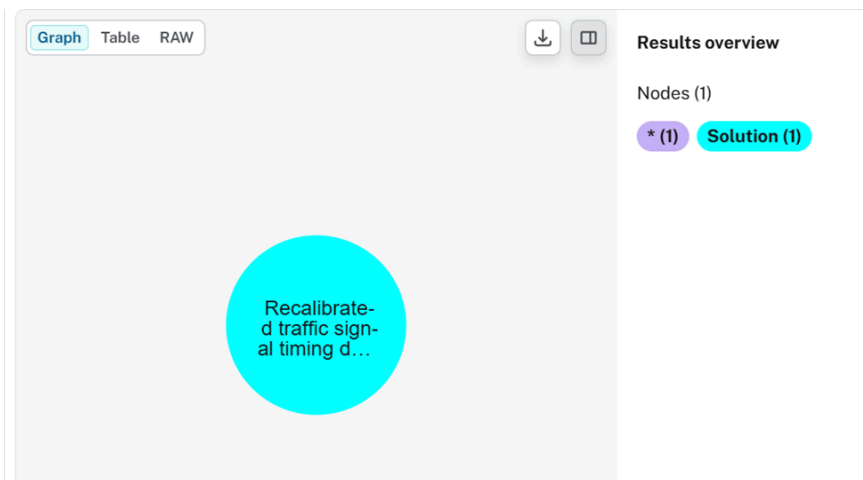
SolutionID	Description	Cost
19	"Road painting crew repainted all lines overnight"	25000.0

16. Citizen reports an issue

```
MATCH (c:Citizen {thai_id: '1100701234580'})
CREATE (i:Issue
        { issue_id: 51,
        category: 'Crimes',
        note: 'Suspicious activity at night market in Khlong Luang', media:
        'https://example.com/images/new_report.jpg',
        longitude: 100.6301,
        latitude: 14.0351, status: 'pending', report_date: date() })
CREATE (c)-[r:REPORTS]->(i)
RETURN c, r, i
```

17. Find a solution that has the highest number of feedback.

```
MATCH (s:Solution)<-[:RATES]-(c:Citizen)
WITH s, COUNT(c) AS feedback_count
ORDER BY feedback_count DESC
LIMIT 1
RETURN s, feedback_count;
```



s	feedback_count
(:Solution {cost: 5000.0, description: "Recalibrated traffic signal timing during peak hours", solution_id: 14})	2

18. How many citizens rated solutions per month

```
MATCH (c:Citizen)-[r:RATES]->(s:Solution)
WITH date(r.feedback_date) AS dt, c
RETURN dt.year AS year, dt.month AS month, COUNT(DISTINCT c) AS
citizens_who_rated
ORDER BY year, month;
```

Table RAW

	year	month	citizens_who_rate
1	2025	8	6
2	2025	9	3
3	2025	10	3

19. Find the top 10 citizens who have both reported the most issues and provided the most feedback in the current year (order by sum of total activity)

```
MATCH (c:Citizen)
OPTIONAL MATCH (c)-[:REPORTS]->(i:Issue)
WHERE i.report_date.year = 2025 WITH c, COUNT(i) AS issuesReported
OPTIONAL MATCH (c)-[f:RATES]->(s:Solution)
WHERE f.feedback_date.year = 2025
WITH c, issuesReported, COUNT(f) AS feedbackGiven
RETURN c.first_name + ' ' + c.last_name AS citizen,          issuesReported,
feedbackGiven,          (issuesReported + feedbackGiven)
AS totalActivity
ORDER BY totalActivity DESC LIMIT 10;
```

Table RAW

	citizen	issuesReported	feedbackGiven	totalActivity
1	"Somchai Boonme e"	3	0	3
2	"Boon-Nam Srisu wan"	2	1	3
3	"Malee Saengkae w"	2	0	2
4	"Apinya Rattan a"	2	0	2
5	"Narong Chokde e"	2	0	2
6	"Anchali Suwann arat"	2	0	2
7	"Panya Kasem"	2	0	2

20. Find the total costs of finished solutions provided to each office in each month

```
MATCH (o:Office)-[g:GIVES]->(s:Solution)
WHERE g.finish_date IS NOT NULL AND s.cost IS NOT NULL
WITH o.name AS office,
        g.finish_date AS finishDate,
        s.cost AS cost
WITH office,
        date(finishDate).year AS year,
        date(finishDate).month AS month,
        SUM(cost) AS totalCost
RETURN office, year, month, totalCost
ORDER BY office, year, month;
```

Table RAW

	office	year	month	totalCost
1	"Khlong Luang District Office"	2025	8	1500.0
2	"Khlong Luang District Office"	2025	10	9500.0
3	"Lam Luk Ka Road Hazard Center"	2025	8	8500.0
4	"Mueang Pathum Thani Public Health Office"	2025	9	750.0
5	"Pathum Thani City Hall"	2025	9	4000.0
6	"Pathum Thani Provincial Elect	2025	8	12000.0